MLOps Assignment 2

Tracking a Pre-trained Sentiment Model with MLflow

Detail	Information
Name	Divya K N
USN	4GW23CI012
Subject	MLOps (Assignment 2)
Model	distilbert-base-uncased-finetuned-sst-2-english (inference only)
Tools	Python, MLflow 3.17.2, Hugging Face Transformers, PyTorch, pandas
Environment	Windows, PowerShell, virtual environment mlops-env
Date	31 October 2026

1. Introduction

MLOps applies the discipline of software engineering to machine learning so that experiments can be repeated, compared and audited. A basic building block is experiment tracking: every execution of a model is stored together with the settings that produced it and the measurements it achieved. MLflow is an open-source platform that offers this record-keeping through a Python API and a web interface.

1.1 MLflow Concepts Used

Concept	Meaning in this assignment
Experiment	A named group of related runs. Here it is sentiment-inference-lab.
Run	One execution of the inference code. Three runs were created: run_batch1, run_batch8 and run_batch4.
Parameter	An input setting recorded once per run: model_name, batch_size and num_samples.
Metric	A numeric result recorded for the run: accuracy, avg_confidence and total_latency_sec.
Artifact	A file stored with the run. The predictions of every run are saved as predictions.csv.

1.2 Python Packages

Package	Purpose
mlflow	Experiment tracking and the MLflow web interface
transformers	Loads the ready-made Hugging Face sentiment model
torch	The deep-learning engine on which the model runs
pandas	Builds the predictions.csv artifact

2. Objective

The aim is to track a pre-trained model with MLflow without any training. The same sentiment classifier is run on a small labelled dataset with different batch sizes, and each execution logs its parameters, metrics and output file. The runs are then compared in the MLflow interface to find out which one performs better.

3. Environment Setup

3.1 Virtual Environment and Packages

An isolated Python environment named mlops-env was created and activated, and the four packages were installed into it. The prompt in Figure 1 starts with (mlops-env), which shows that the environment was active.

```
python -m venv mlops-env
mlops-env\Scripts\activate
python -m pip install --upgrade pip
pip install mlflow transformers torch pandas
```

3.2 Verification and Model Download

The installation was verified by importing all four packages and printing the MLflow version, which is 3.17.2 (visible in the sidebar of Figure 2). The model was then downloaded once so that later runs could use the cached copy.

```
python -c "import mlflow, transformers, torch, pandas;
print(mlflow.__version__)"
python -c "from transformers import pipeline; pipeline('sentiment-analysis',
model='distilbert-base-uncased-finetuned-sst-2-english')"
```

During the download (Figure 1), the files config.json, model.safetensors (268 MB), tokenizer_config.json and vocab.txt were fetched. Two warnings appeared: one about unauthenticated requests to the Hugging Face Hub and one about symbolic links not being supported on Windows. Neither affects the results; the second only means that the cache uses a little more disk space.

4. Methodology

4.1 Dataset

A small labelled dataset of ten sentences, five positive and five negative, was used. The label next to each sentence is the expected sentiment.

No.	Sentence	Expected label
1	I loved this movie, it was fantastic!	POSITIVE
2	The service was terrible and slow.	NEGATIVE
3	What a wonderful experience.	POSITIVE
4	I regret buying this product.	NEGATIVE
5	The food was delicious and fresh.	POSITIVE
6	This is the worst app I have ever used.	NEGATIVE
7	Absolutely brilliant performance by the team.	POSITIVE
8	The battery dies within an hour.	NEGATIVE
9	Great value for money, highly recommend.	POSITIVE
10	I am very disappointed with the quality.	NEGATIVE

4.2 Experiment Design

Only the batch size was changed between runs. The model, the sentences and the code were identical, so any difference in latency comes from batching alone and any difference in accuracy would indicate a problem.

Run name	Model	Batch size
run_batch1	distilbert-base-uncased-finetuned-sst-2-english	1
run_batch8	distilbert-base-uncased-finetuned-sst-2-english	8
run_batch4	distilbert-base-uncased-finetuned-sst-2-english	4

4.3 Information Logged for Every Run

Type	Name	Description
Parameter	model_name	Name of the Hugging Face model
Parameter	batch_size	Number of sentences processed together
Parameter	num_samples	Number of sentences (10)
Metric	accuracy	Fraction of predictions that match the expected label
Metric	avg_confidence	Mean confidence score of the predictions
Metric	total_latency_sec	Time taken by the inference call for all sentences
Artifact	predictions.csv	Text, expected label, predicted label and confidence

5. Implementation

The code is saved as mlflow_lab.py. It has three parts: the dataset and experiment name, the run_experiment function, and the calls that create the runs.

5.1 Dataset and Experiment

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE"),
]
texts = [t for t, _ in data]
labels = [l for _, l in data]

mlflow.set_experiment("sentiment-inference-lab")
```

5.2 The run_experiment Function

```
def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)

    with mlflow.start_run(run_name=run_name):
        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [p["label"] for p in preds]
        accuracy = sum(p == l for p, l in zip(pred_labels, labels)) / len(labels)
        avg_conf = sum(p["score"] for p in preds) / len(preds)

        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_conf)
        mlflow.log_metric("total_latency_sec", latency)
```

```
        # Artifact
        df = pd.DataFrame({
            "text": texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [p["score"] for p in preds],
        })
        df.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))
```

5.3 Creating the Runs

```
MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")  # Task 3: third run
```

Key points of the implementation:

- **Timing:** time.time() surrounds only the call to the classifier, so model loading is excluded from total_latency_sec.
- **Accuracy:** predicted labels are compared with the expected labels and the number of matches is divided by ten.
- **Confidence:** avg_confidence is the mean of the score returned for each sentence.
- **Artifact:** a pandas DataFrame is written to predictions.csv and attached to the active run with mlflow.log_artifact.
- **Third run:** the last line adds run_batch4 with the same model and batch_size=4, as required by Task 3.

6. Execution

6.1 Running the Script

The script was started from PowerShell inside the project folder.

```
python mlflow_lab.py
```

On the first execution MLflow created its local tracking database and then created the experiment sentiment-inference-lab, as the log lines in Figure 1 show. Each run reloaded the model weights and printed one result line:

Run	Batch size	Accuracy	Latency (s)
run_batch1	1	1.0	0.129
run_batch8	8	1.0	0.048
run_batch4	4	1.0	0.044

6.2 Starting the MLflow Interface

The tracking interface was opened in the browser at http://127.0.0.1:5000. The home page lists the experiment sentiment-inference-lab together with the default experiment, both created on 31 October 2026 at 12:03 PM (Figure 2).

6.3 Comparing the Runs

Inside the experiment, the Evaluation runs view lists run_batch4, run_batch8 and run_batch1, and all three show a finished status. The Metrics section draws one bar chart per metric (Figure 3). The accuracy and avg_confidence charts show 1.00 for every run, while the total_latency_sec chart shows clearly different bars: about 0.04 s for run_batch4, 0.05 s for run_batch8 and 0.13 s for run_batch1.

6.4 Inspecting the Artifact

Opening run_batch4 and choosing the Artifacts tab shows predictions.csv (784 bytes). The preview lists the text, actual, predicted and confidence columns (Figure 4). In every visible row the predicted label equals the actual label, and the confidence scores lie between about 0.998 and 0.9999.

7. Results and Discussion

7.1 Summary of Results

The speed-up column compares each run with run_batch1, which is the slowest.

Run	Batch size	Correct	Accuracy	Latency (s)	Speed-up
run_batch1	1	10 / 10	1.0	0.129	1.0x
run_batch8	8	10 / 10	1.0	0.048	2.7x
run_batch4	4	10 / 10	1.0	0.044	2.9x

7.2 Short Report (Task 4)

1. Which run had the lowest latency, and why might that be?

run_batch4 had the lowest latency (0.044 s), followed by run_batch8 (0.048 s) and run_batch1 (0.129 s). Processing several sentences together reduces the repeated overhead of calling the model, so run_batch1, which makes ten separate calls, was almost three times slower. The difference between batch sizes 4 and 8 is only 0.004 s, which is small enough to change between executions because of warm-up and system load.

2. Did accuracy change between runs? Why or why not?

No. All three runs classified 10 of 10 sentences correctly, so the accuracy stayed at 1.0. The same pre-trained model and the same sentences were used, and the batch size only changes how the inputs are grouped for inference. It does not change the model weights or the predictions.

3. Name two benefits of using MLflow instead of printing results in the terminal.

- **Reproducibility and record-keeping:** MLflow stores the parameters, metrics and artifacts such as predictions.csv for every run, so the work can be inspected again later.
- **Easy comparison:** MLflow shows the runs and their metrics side by side in charts, which makes accuracy, confidence and latency easier to compare than lines of terminal output.

8. Conclusion

A pre-trained DistilBERT sentiment model was run on ten labelled sentences with batch sizes 1, 8 and 4, and every run was tracked with MLflow. Accuracy was 100% in all runs, while latency fell from 0.129 s with batch size 1 to 0.044 s with batch size 4, so batching gave the best speed without any loss of accuracy. The MLflow interface made it simple to compare the runs and to inspect the saved predictions. The screenshots supporting each step are given in the annexure that follows.

Annexure: Execution Screenshots

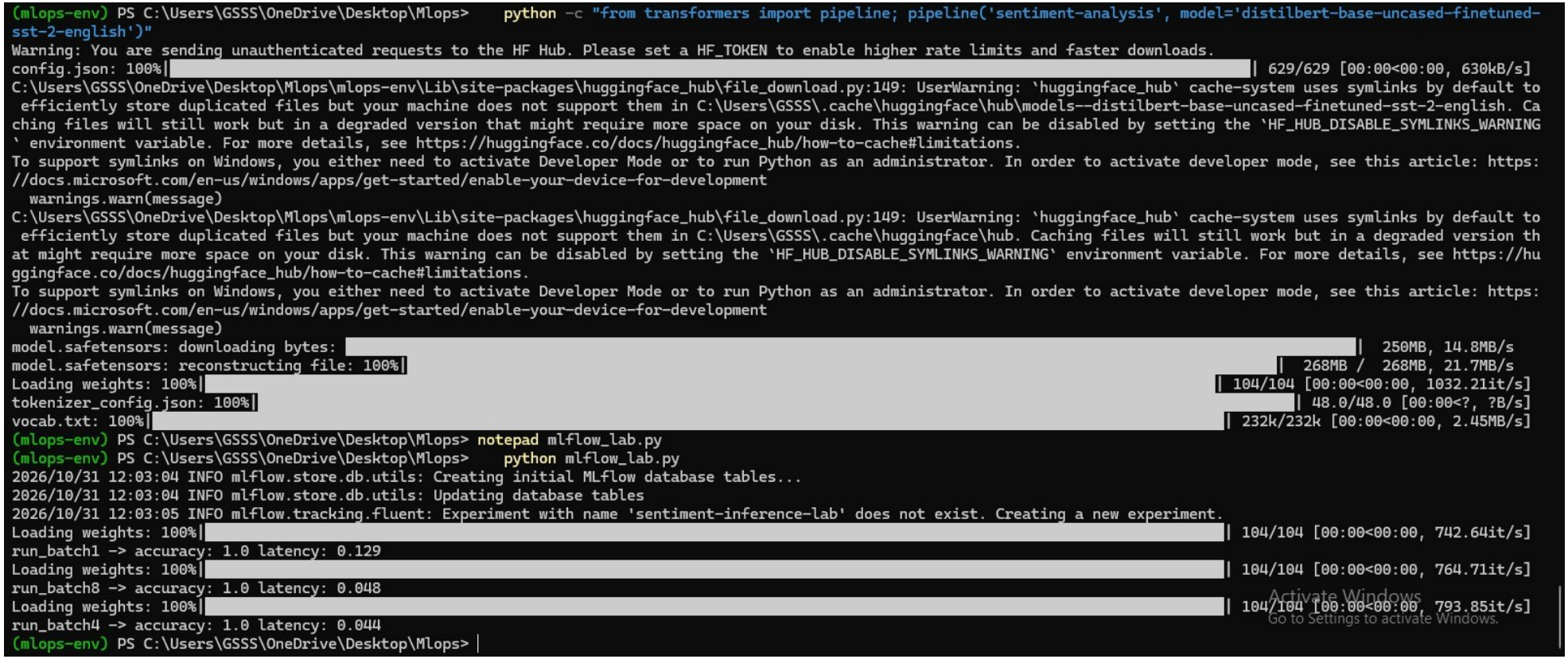


Figure 1: Model download and execution of mlflow_lab.py with the three runs

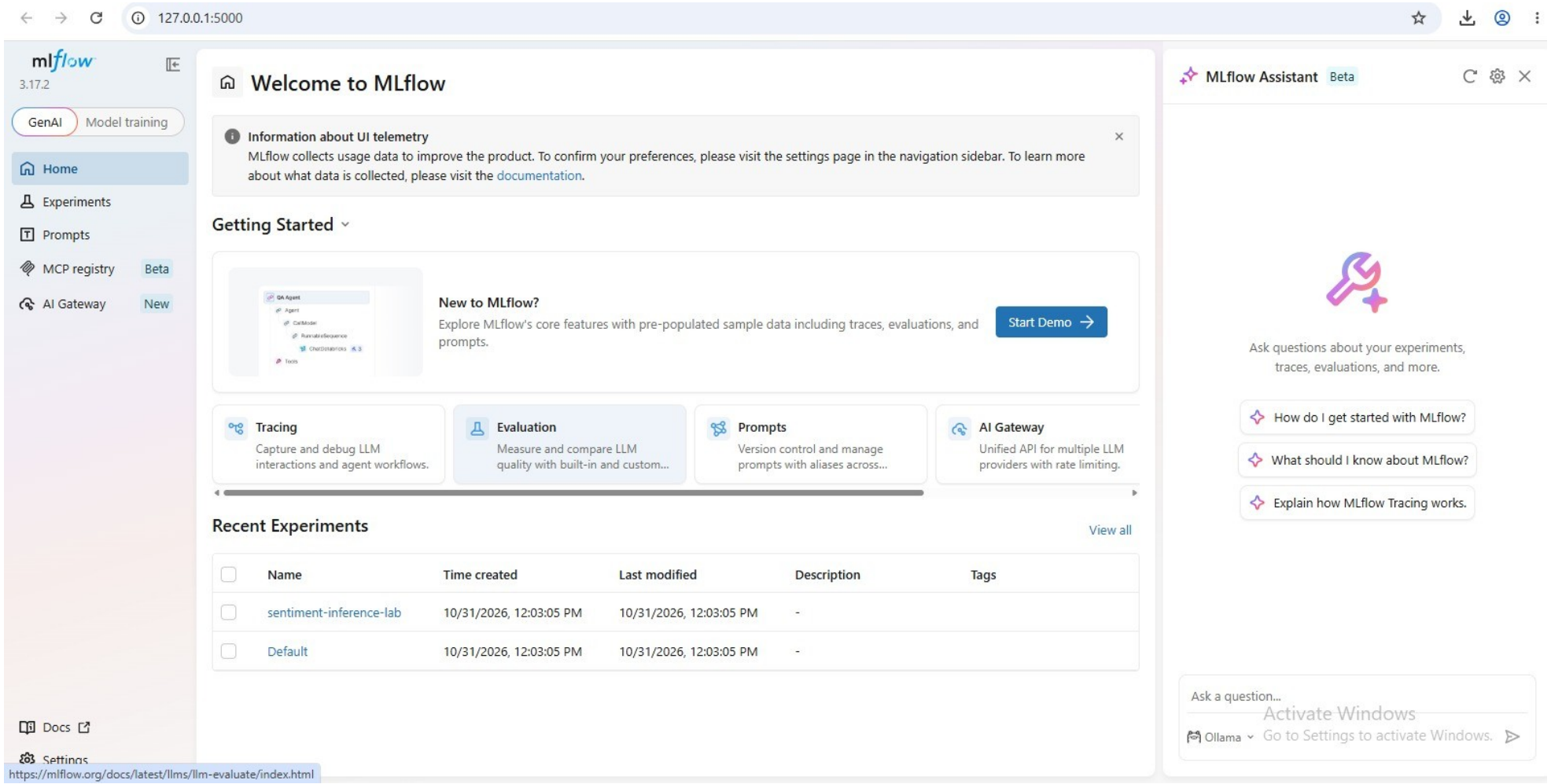


Figure 2: MLflow home page showing the sentiment-inference-lab experiment

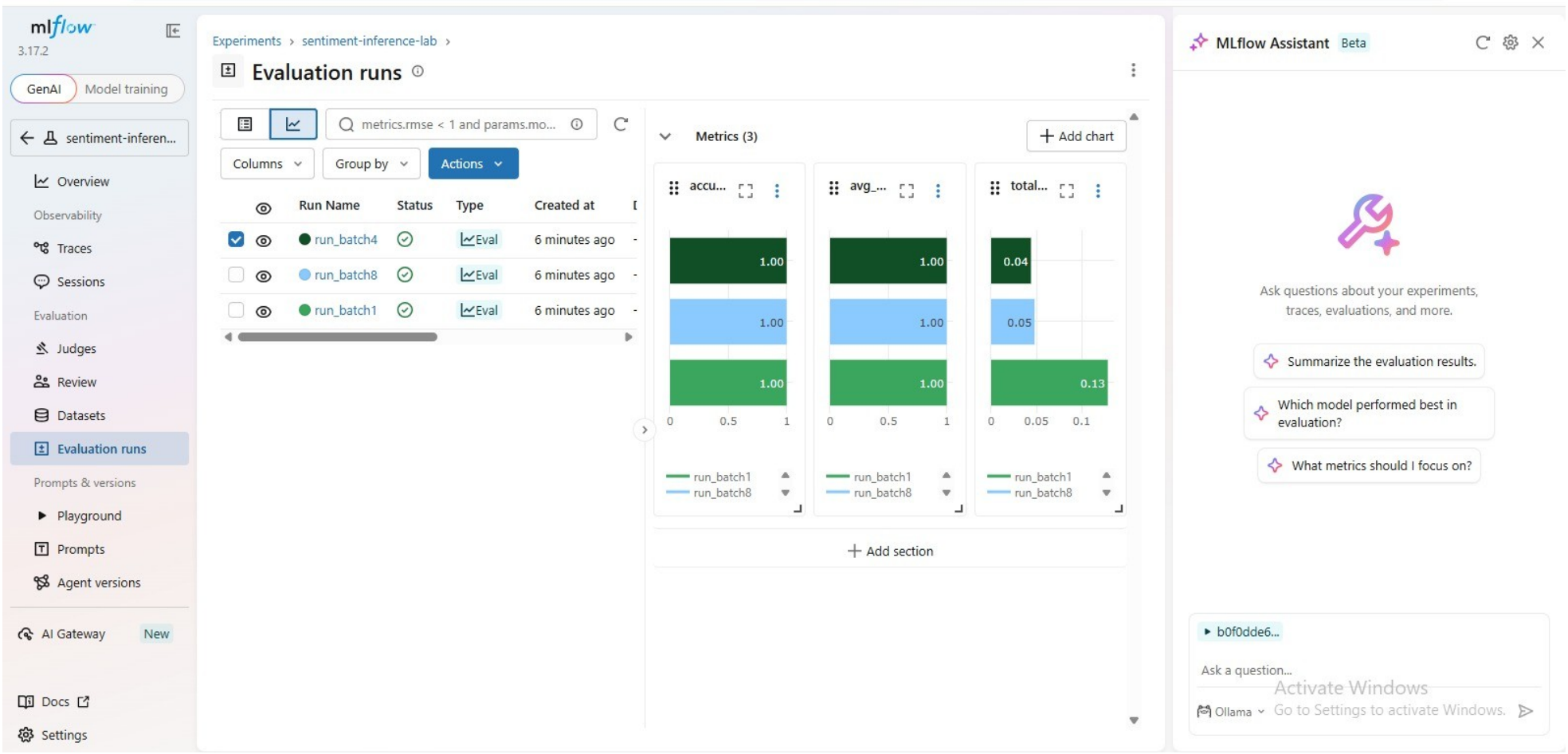


Figure 3: Runs table and metric comparison charts (accuracy, avg_confidence, total_latency_sec)

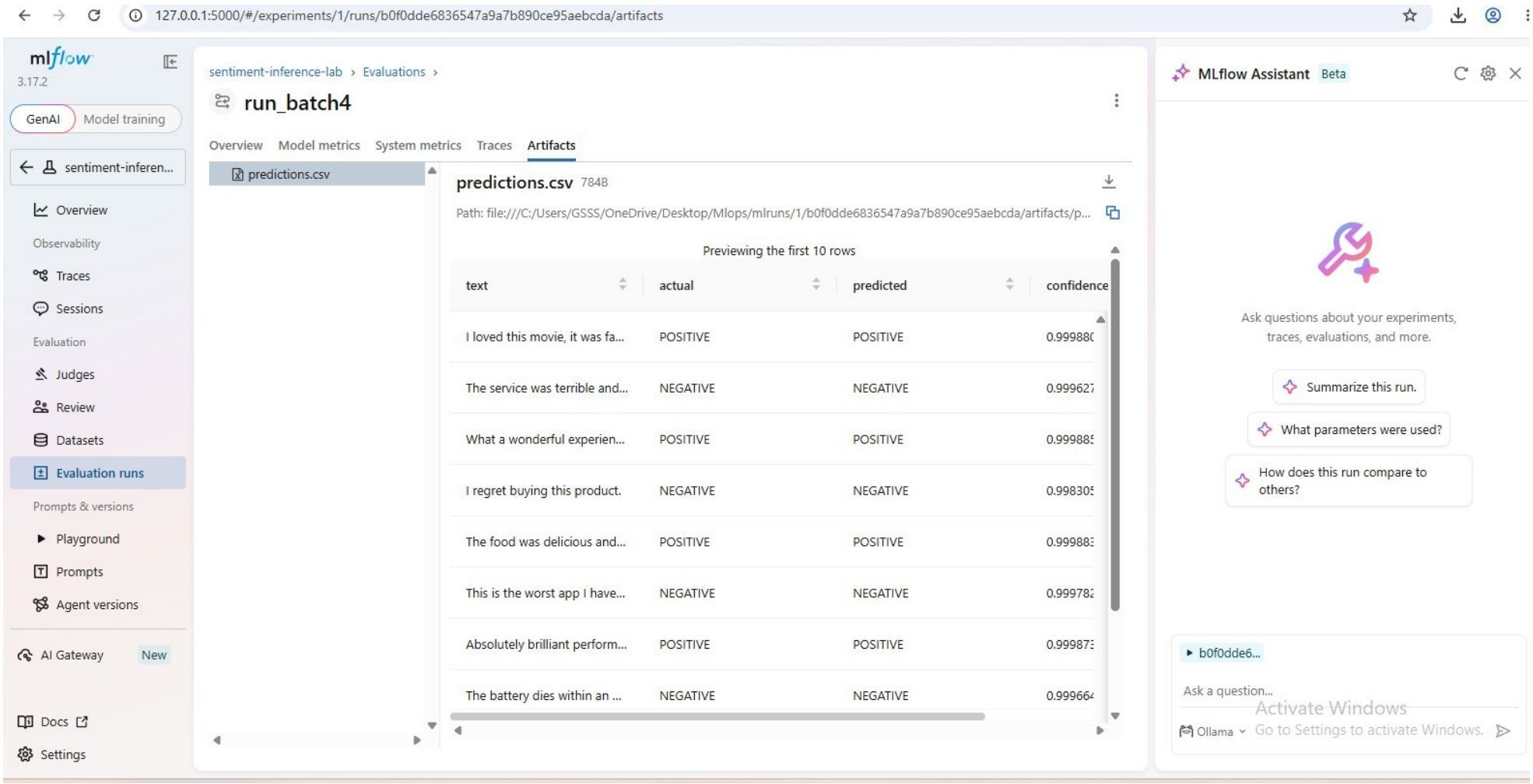


Figure 4: predictions.csv artifact of run_batch4